

EXPT NO: 4**DATE: 17-08-2026****TRANSFER LEARNING USING PRE-TRAINED VISION MODELS FOR
IMAGE RECOGNITION****PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)**

1. What is transfer learning in deep learning?

Transfer learning is a deep learning technique in which a model that has already been trained on a large dataset is reused for solving a different but related problem. The pre-trained model has already learned useful features such as edges, shapes, textures, and patterns from the original dataset. Instead of starting the training process with random weights, the learned knowledge is transferred to the new task. This makes the development process faster and more efficient.

For example, a model trained on ImageNet can be reused for classifying images of different types of plants or animals. The final layers of the model can be modified according to the new classification task, while the earlier layers can retain the features they have already learned. Thus, transfer learning is especially useful when the new dataset is relatively small

2. What are the advantages of using pre-trained models over training from scratch?
Pre-trained models have already learned important features from large and diverse datasets. Therefore, they can be adapted to a new problem without requiring the entire model to be trained from the beginning. This saves considerable training time and computational resources while often providing better performance when the available dataset is limited.

Advantage	Description
Reduced training time	The model already contains learned weights, so less training is required.
Less data required	Useful features have already been learned from a large dataset.
Lower computational cost	Training from scratch generally requires more processing power and time.
Better performance	Pre-trained models can provide a strong starting point for the new task.
Easy implementation	Existing architectures and weights can be directly reused and fine-tuned.

3. What is the role of fine-tuning in transfer learning?

Fine-tuning is the process of taking a pre-trained model and training it further on a new dataset related to the target problem. During fine-tuning, some or all of the model's layers may be allowed to update their weights. Usually, a smaller learning rate is used so that the useful knowledge already present in the model is not destroyed.

The main purpose of fine-tuning is to make the pre-trained model more suitable for the specific task. For example, a model trained on general images can be fine-tuned using a dataset containing different types of medical images or industrial objects. This

allows the model to retain its general feature-learning ability while adapting to the characteristics of the new dataset.

4. Name any three popular pre-trained computer vision models.

Some commonly used pre-trained computer vision models are:

VGGNet – A convolutional neural network known for its simple architecture using multiple convolutional layers. It is commonly used for image classification and feature extraction.

ResNet – A deep convolutional neural network that uses residual connections to solve the vanishing-gradient problem and allows very deep networks to be trained effectively.

EfficientNet – A family of models designed to achieve good accuracy while maintaining computational efficiency by systematically scaling the network's depth, width, and input resolution.

These models are commonly available with pre-trained weights, particularly weights learned from large datasets such as ImageNet, and can be adapted for various computer vision applications.

5. What is the purpose of Hugging Face and how is it useful in deep learning?

Hugging Face is an open-source platform and AI ecosystem that provides access to a large collection of pre-trained models, datasets, and development libraries. It is widely known for natural language processing tasks but also supports computer vision, audio processing, and multimodal deep learning applications. Its libraries allow developers to easily download and use existing models instead of implementing and training them completely from scratch.

Hugging Face is particularly useful because it provides tools for model selection, inference, fine-tuning, dataset handling, and model sharing. Its Transformers library contains many popular pre-trained models that can be adapted to specific applications. This makes experimentation easier and reduces the time and computational resources required to develop deep learning solutions.

AIM

To implement transfer learning using pre-trained vision models for image recognition and evaluate their performance on real-world image datasets.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow/Keras or PyTorch
- Google Colab
- Hugging Face Transformers
- Kaggle Dataset
- Matplotlib
- GitHub

OBJECTIVES

Students will be able to:

- Understand the concept of transfer learning.
- Apply pre-trained vision models for image recognition.
- Fine-tune models such as ResNet or AlexNet.

- Utilize Kaggle datasets and Hugging Face vision models.
- Compare transfer learning performance with custom CNN models.

PROCEDURE

PART A: DATASET PREPARATION

Perform the following tasks:

- Download an image dataset from Kaggle.
- Load and preprocess the images.
- Split the dataset into training, validation, and testing sets.

TRANSFER LEARNING USING PRE-TRAINED VISION MODELS FOR IMAGE RECOGNITION

by Rohith John 24BAD100

Part A: Dataset Preparation

Importing Dog and Cat dataset from local storage

```
1: import os
import tensorflow as tf
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
TRAIN_PATH = r"/kaggle/input/datasets/jakupymera3/cats-and-dogs-image-dataset/dataset/training_set"
TEST_PATH = r"/kaggle/input/datasets/jakupymera3/cats-and-dogs-image-dataset/dataset/test_set"
print("Training folder:", TRAIN_PATH)
print("Testing folder:", TEST_PATH)
```

Training folder: /kaggle/input/datasets/jakupymera3/cats-and-dogs-image-dataset/dataset/training_set
Testing folder: /kaggle/input/datasets/jakupymera3/cats-and-dogs-image-dataset/dataset/test_set

Splitting Dataset in train, test and validation sets:

```
1: import tensorflow as tf
import os
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
IMG_SIZE = (224, 224)
BATCH_SIZE = 16
SEED = 42

train_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_PATH,
    validation_split=0.20,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_PATH,
    validation_split=0.20,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary"
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    TEST_PATH,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary",
    shuffle=False
)
```

Found 8000 files belonging to 2 classes.
Using 6400 files for training.
2026-08-16 17:08:33.008990: E external/local_xla/xla/stream_executor/cuda/cuda_platf
Found 8000 files belonging to 2 classes.
Using 1600 files for validation.
Found 2000 files belonging to 2 classes.

```
1: #Check class names
print("Class names:", train_ds.class_names)
```

Class names: ['cats', 'dogs']

PART B: IMPLEMENTING TRANSFER LEARNING

Load a pre-trained model such as:

- ResNet-50
- AlexNet

Replace the final classification layer according to the dataset classes.

Freeze the feature extraction layers.
Compile the model.

/RohithJohnM/24ADI006-Deep-Learning/blob/main/Exp%204/Transfer%20Learning%20using%20Pre-trained%20models%2024BAD100.ipynb

24ADI006-Deep-Learning / Exp 4 / Transfer Learning using Pre-trained models 24BAD100.ipynb

PreviewCodeBlame1 lines (1 loc) · 495 KB

#Check class names
print("Class names:", train_ds.class_names)

Class names: ['cats', 'dogs']

In [4]:
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
test_ds = test_ds.prefetch(buffer_size=AUTOTUNE)

print("Dataset preprocessing completed successfully.")

Dataset preprocessing completed successfully.

PART B: IMPLEMENTING TRANSFER LEARNING

Loading the Pre-trained ResNet-50 Model

In [8]:
from tensorflow.keras.applications import ResNet50

base_model = ResNet50(
 weights="/kaggle/input/models/mrohithjohn/resnet50/keras/default/1/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5",
 include_top=False,
 input_shape=(224, 224, 3)
)

print("ResNet-50 loaded successfully.")

ResNet-50 loaded successfully.

Freezing the Feature Extraction Layers

In [9]:
base_model.trainable = False
print("Base model trainable:", base_model.trainable)

Base model trainable: False

24ADI006-Deep-Learning / Exp 4 / Transfer Learning using Pre-trained models 24BAD100.ipynb

PreviewCodeBlame1 lines (1 loc) · 495 KB

Replacing the Final Classification Layer

In [10]:
from tensorflow.keras import layers, models

model = models.Sequential([
 base_model,
 layers.GlobalAveragePooling2D(),
 layers.Dense(128, activation="relu"),
 layers.Dropout(0.3),
 layers.Dense(1, activation="sigmoid")
)

model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 23,850,113 (90.98 MB)
Trainable params: 262,401 (1.00 MB)
Non-trainable params: 23,587,712 (89.98 MB)

Compiling the Transfer Learning Model

In [11]:
model.compile(
 optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
 loss="binary_crossentropy",
 metrics=["accuracy"]
)

print("Model compiled successfully.")

Model compiled successfully.

PART C: MODEL TRAINING AND PERFORMANCE EVALUATION

Train the transfer learning model.

Record:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss
- Test accuracy

PART C: MODEL TRAINING AND PERFORMANCE EVALUATION

Training the Transfer Learning Model

```
EPOCHS = 5
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
```

```
Epoch 1/5
400/400 ————— 490s 1s/step - accuracy: 0.9458 - loss: 0.1372 - val_accuracy: 0.9769 - val_loss: 0.0617
Epoch 2/5
400/400 ————— 535s 1s/step - accuracy: 0.9794 - loss: 0.0553 - val_accuracy: 0.9762 - val_loss: 0.0549
Epoch 3/5
400/400 ————— 485s 1s/step - accuracy: 0.9834 - loss: 0.0440 - val_accuracy: 0.9781 - val_loss: 0.0538
Epoch 4/5
400/400 ————— 502s 1s/step - accuracy: 0.9858 - loss: 0.0356 - val_accuracy: 0.9769 - val_loss: 0.0510
Epoch 5/5
400/400 ————— 531s 1s/step - accuracy: 0.9905 - loss: 0.0270 - val_accuracy: 0.9819 - val_loss: 0.0530
```

Recording Training and Validation Performance

```
history_df = pd.DataFrame(history.history)
print(history_df)
```

	accuracy	loss	val_accuracy	val_loss
0	0.945781	0.137220	0.976875	0.061687
1	0.979375	0.055323	0.976250	0.054891
2	0.983437	0.044038	0.978125	0.053793
3	0.985781	0.035587	0.976875	0.050974
4	0.990469	0.026975	0.981875	0.052971

Accuracy vs Epoch

```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    marker="o",
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)

plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
```

Preview Code Blame 1 lines (1 loc) · 495 KB

Accuracy vs Epoch

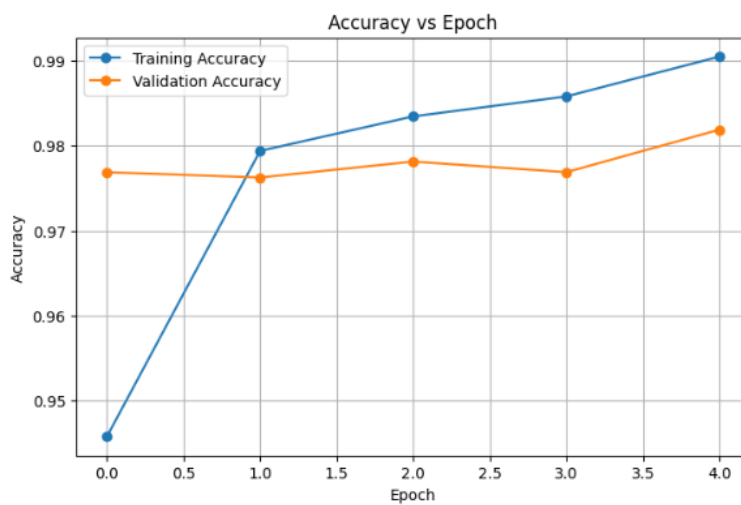
```
In [14]: plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    marker="o",
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)

plt.title("Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Loss vs Epoch

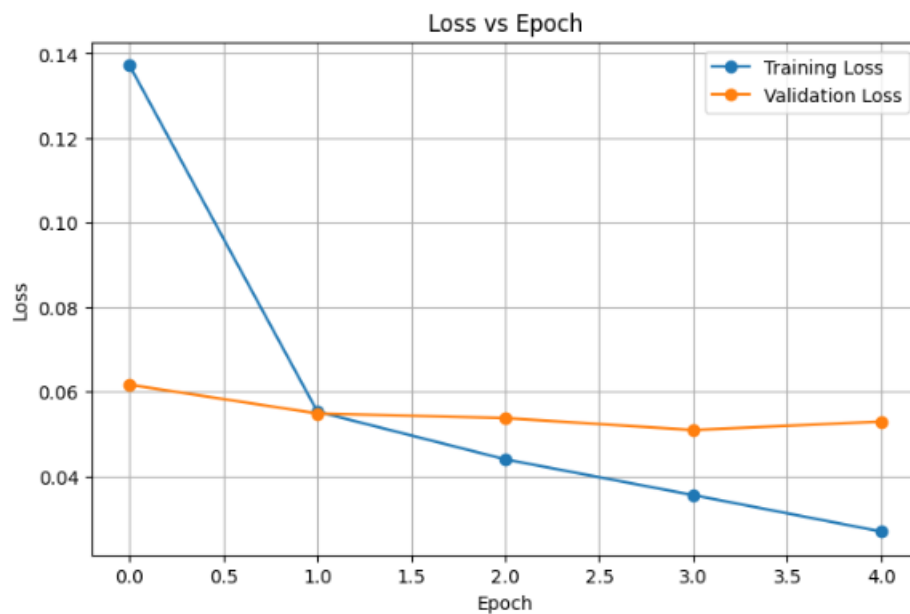
```
In [15]: plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    marker="o",
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    marker="o",
    label="Validation Loss"
)

plt.title("Loss vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Preview Code Blame 1 lines (1 loc) · 495 KB

Evaluating the Model on the Test Dataset

```
In [16]: test_loss, test_accuracy = model.evaluate(test_ds)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
print("Test Accuracy (%)ate", test_accuracy * 100)
```

```
125/125 ————— 123s 980ms/step - accuracy: 0.9820 - loss: 0.0541
Test Loss: 0.0541052855512428
Test Accuracy: 0.9819999933242798
Test Accuracy (%): 98.19999933242798
```

```
In [70]: # Recreate the TensorFlow Transfer Learning Model

from tensorflow.keras.applications import ResNet50
from tensorflow.keras import layers, models

tf_base_model = ResNet50(
    weights="/kaggle/input/models/mrohithjohn/resnet50/keras/default/1/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5",
    include_top=False,
    input_shape=(224, 224, 3)
)

# Freeze ResNet-50 feature extraction layers
tf_base_model.trainable = False

# Create the TensorFlow transfer learning model
tf_model = models.Sequential([
    tf_base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid")
])

# Compile
tf_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("TensorFlow Transfer Learning Model recreated successfully.")
```

```
TensorFlow Transfer Learning Model recreated successfully.
```

Visualize:

- Accuracy vs Epoch
- Loss vs Epoch

PART D: USING HUGGING FACE PRE-TRAINED MODELS

Perform the following tasks:

- Load a pre-trained vision model from Hugging Face.
- Perform image classification on sample images.
- Compare predictions with the transfer learning model.
- Document observations and upload the notebook to GitHub.

Preview Code Blame 1 lines (1 loc) · 495 KB

PART D: USING HUGGING FACE PRE-TRAINED MODELS

```
!pip install -U transformers huggingface_hub pillow
```

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
 WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken (WARNING: Retrying (Retry(total=5, connect=None, read=None, redirect=None, status=None)) after connection broken ^C

```
!python3
import os
import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from PIL import Image
from torch.utils.data import Dataset, DataLoader
from transformers import AutoImageProcessor, ResNetForImageClassification

MODEL_PATH = "/kaggle/input/datasets/erohithjohn/huggingface-resnet50-weight"

DATASET_PATH = "/kaggle/input/datasets/jakupymeraj/cats-and-dogs-image-dataset/dataset"

TRAIN_PATH = os.path.join(DATASET_PATH, "training_set")
TEST_PATH = os.path.join(DATASET_PATH, "test_set")

print("Model path:", MODEL_PATH)
print("Training path:", TRAIN_PATH)
print("Testing path:", TEST_PATH)
```

Model path: /kaggle/input/datasets/erohithjohn/huggingface-resnet50-weight
 Training path: /kaggle/input/datasets/jakupymeraj/cats-and-dogs-image-dataset/dataset/training_set
 Testing path: /kaggle/input/datasets/jakupymeraj/cats-and-dogs-image-dataset/dataset/test_set

Load Hugging Face processor

```
!python3
processor = AutoImageProcessor.from_pretrained(
    MODEL_PATH,
    local_files_only=True
)

print("Image processor loaded successfully.")
print(processor)

Image processor loaded successfully.
ConvNextImageProcessorFast {
  "crop_pct": 0.875,
  "data_format": "channels_first",
  "do_normalize": true,
  "do_rescale": true,
  "do_resize": true,
  "image_mean": [
    0.485,
    0.456,
    0.406
  ],
  "image_processor_type": "ConvNextImageProcessorFast",
  "image_std": [
    0.229,
    0.224,
    0.225
  ],
  "resample": 3,
  "rescale_factor": 0.003921568627450988,
  "size": {
    "shortest_edge": 224
  }
}
```

Load ResNet-50 and change it to 2 classes

```
!python3
hf_model = ResNetForImageClassification.from_pretrained(
    MODEL_PATH,
    num_labels=2,
    ignore_mismatched_sizes=True
)
```

Load ResNet-50 and change it to 2 classes

```
In [46]: hf_model = ResNetForImageClassification.from_pretrained(
    MODEL_PATH,
    num_labels=2,
    ignore_mismatched_sizes=True,
    local_files_only=True
)

hf_model.config.id2label = {
    0: "CAT",
    1: "DOG"
}

hf_model.config.label2id = {
    "CAT": 0,
    "DOG": 1
}

print("Hugging Face ResNet-50 loaded successfully.")
print("Number of classes:", hf_model.config.num_labels)

Loading weights: OK | 0/120 [00:00<?, ?it/s]
ResNetForImageClassification LOAD REPORT from: /kaggle/input/datasets/erohithjohn/huggingface-resnet50-weight
Key | Status |
-----|-----|
classifier.1.weight | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([1000, 2048]) vs model:torch.
classifier.1.bias | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([1000]) vs model:torch.Size([
Notes:
- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.
Hugging Face ResNet-50 loaded successfully.
Number of classes: 2
```

Loading the Hugging Face Pre-trained ResNet-50 Model

Prepare the Dataset for PyTorch

```
In [47]: from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import torch

# Image preprocessing
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Load training and testing datasets
hf_train_dataset = datasets.ImageFolder(
    TRAIN_PATH,
    transform=transform
)

hf_test_dataset = datasets.ImageFolder(
    TEST_PATH,
    transform=transform
)

# Create DataLoaders
hf_train_loader = DataLoader(
    hf_train_dataset,
    batch_size=16,
    shuffle=True,
    num_workers=2
)

hf_test_loader = DataLoader(
    hf_test_dataset,
    batch_size=16,
    shuffle=False,
    num_workers=1
)
```

Preview Code Blame 1 lines (1 loc) · 495 KB

```
print("Classes:", hf_train_dataset.classes)
print("Training images:", len(hf_train_dataset))
print("Testing images:", len(hf_test_dataset))
```

Classes: ['cats', 'dogs']
 Training images: 8000
 Testing images: 2000

Freezing the Pre-trained Feature Extraction Layers

In [48]:

```
# Freeze the pre-trained ResNet-50 backbone
for param in hf_model.resnet.parameters():
    param.requires_grad = False

# Keep only the new classification layer trainable
for param in hf_model.classifier.parameters():
    param.requires_grad = True

# Display trainable parameters
trainable_params = sum(
    p.numel() for p in hf_model.parameters() if p.requires_grad
)

total_params = sum(
    p.numel() for p in hf_model.parameters()
)

print("Total parameters:", total_params)
print("Trainable parameters:", trainable_params)
```

Total parameters: 23512138
 Trainable parameters: 4898

Training the Hugging Face ResNet-50 Model

In [49]:

```
import torch.nn as nn
import torch.optim as optim

# Select device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Using device:", device)

hf_model = hf_model.to(device)

# Loss function
criterion = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.Adam(
    hf_model.classifier.parameters(),
    lr=0.001
)

EPOCHS_HF = 5

train_losses = []
train_accuracies = []

for epoch in range(EPOCHS_HF):
    hf_model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in hf_train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = hf_model(pixel_values=images)
        loss = criterion(outputs.logits, labels)
        loss.backward()
```

Preview Code Blame 1 lines (1 loc) · 495 KB

```
optimizer.step()

running_loss += loss.item()

predictions = torch.argmax(outputs.logits, dim=1)
correct += (predictions == labels).sum().item()
total += labels.size(0)

epoch_loss = running_loss / len(hf_train_loader)
epoch_accuracy = correct / total

train_losses.append(epoch_loss)
train_accuracies.append(epoch_accuracy)

print(
    f"Epoch (epoch + 1)/(EPOCHS_HF) * "
    f"Loss: (epoch_loss:.4f) * "
    f"Accuracy: (epoch_accuracy:.4f)"
)
```

Using device: cpu
 Epoch 1/5 - Loss: 0.1901 - Accuracy: 0.9501
 Epoch 2/5 - Loss: 0.8965 - Accuracy: 0.9691
 Epoch 3/5 - Loss: 0.8781 - Accuracy: 0.9739
 Epoch 4/5 - Loss: 0.8663 - Accuracy: 0.9759
 Epoch 5/5 - Loss: 0.8651 - Accuracy: 0.9765

Evaluating the Hugging Face Model on the Test Dataset

In [51]:

```
hf_model.eval()

correct = 0
total = 0

with torch.no_grad():
    for images, labels in hf_test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = hf_model(pixel_values=images)

        predictions = torch.argmax(outputs.logits, dim=1)
        correct += (predictions == labels).sum().item()
        total += labels.size(0)

hf_test_accuracy = correct / total

print("Hugging Face Test Accuracy:", hf_test_accuracy)
print("Hugging Face Test Accuracy (%):", hf_test_accuracy * 100)
```

Hugging Face Test Accuracy: 0.9855
 Hugging Face Test Accuracy (%): 98.55000000000001

Hugging Face Training Accuracy

In [52]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS_HF + 1),
    train_accuracies,
    marker="o",
    label="Training Accuracy"
)

plt.title("Hugging Face ResNet-50 Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```

Image Classification Using the Hugging Face Model

```
import random
import os
import matplotlib.pyplot as plt
from PIL import Image

# Select a random image from the test dataset
sample_class = random.choice(["cats", "dogs"])

sample_folder = os.path.join(
    TEST_PATH,
    sample_class
)

sample_image_name = random.choice(
    os.listdir(sample_folder)
)

sample_image_path = os.path.join(
    sample_folder,
    sample_image_name
)

# Load image
sample_image = Image.open(
    sample_image_path
).convert("RGB")

# Display image
plt.figure(figsize=(5, 5))
plt.imshow(sample_image)
plt.axis("off")
plt.title("Sample Test Image")
plt.show()

print("Actual Class:", sample_class)
```

Hugging Face Training Accuracy

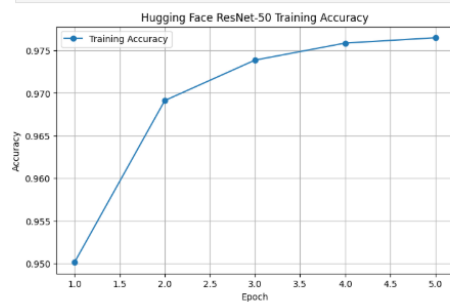
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS_HF + 1),
    train_accuracies,
    marker="o",
    label="Training Accuracy"
)

plt.title("Hugging Face ResNet-50 Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Sample Test Image



Actual Class: cats

Comparing Hugging Face and Transfer Learning Predictions

In [78]:

```
import numpy as np
import tensorflow as tf

# Convert PIL image to Numpy array
tf_image = np.array(sample_image)

# Resize image
tf_image = tf.image.resize(
    tf_image,
    (224, 224)
)

# Convert to float
tf_image = tf.cast(
    tf_image,
    tf.float32
)

# Add batch dimension
tf_image = tf.expand_dims(
    tf_image,
    axis=0
)

# Predict using TensorFlow model
tf_prediction = tf_model.predict(
    tf_image,
    verbose=0
)

# Extract probability
tf_probability = float(
    np.asarray(tf_prediction).reshape(-1)[0]
)

# Convert probability to class
if tf_probability >= 0.5:
    tf_class = "dogs"
    tf_confidence = tf_probability
else:
    tf_class = "cats"
    tf_confidence = 1 - tf_probability

print("Actual Class:", sample_class)

print("\nTensorFlow Transfer Learning Model:")
print("Prediction:", tf_class)
print("Confidence:", round(tf_confidence * 100, 2), "%")

print("\nHugging Face ResNet-50:")
print("Prediction:", predicted_class.lower())
print("Confidence:", round(hf_confidence * 100, 2), "%")
```

Actual Class: cats

TensorFlow Transfer Learning Model:
Prediction: cats
Confidence: 100.0 %Hugging Face ResNet-50:
Prediction: cat
Confidence: 98.59 %

Prediction Comparison

```
import pandas as pd

prediction_comparison = pd.DataFrame({
    "Model": [
        "TensorFlow Transfer Learning ResNet-50",
        "Hugging Face ResNet-50"
    ],
    "Actual Class": [
        sample_class,
        sample_class
    ],
    "Predicted Class": [
        tf_class,
        predicted_class.lower()
    ],
    "Confidence (%)": [
        round(tf_confidence * 100, 2),
        round(hf_confidence * 100, 2)
    ]
})

prediction_comparison
```

	Model	Actual Class	Predicted Class	Confidence (%)
0	TensorFlow Transfer Learning ResNet-50	cats	cats	100.00
1	Hugging Face ResNet-50	cats	cat	98.59

```
# Test accuracy comparison

print(
    f"TensorFlow ResNet-50 Test Accuracy: "
    f"{tf_test_accuracy * 100:.2f}%"
)

print(
    f"Hugging Face ResNet-50 Test Accuracy: "
    f"{hf_test_accuracy * 100:.2f}%"
)
```

TensorFlow ResNet-50 Test Accuracy: 97.85%
Hugging Face ResNet-50 Test Accuracy: 98.55%

Github ipynb link: <https://github.com/RohithJohnM/24ADI006-Deep-Learning/blob/main/Exp%204/Transfer%20Learning%20using%20Pre-trained%20models%2024BAD100.ipynb>

EXPECTED OUTCOME

Students will understand the advantages of transfer learning and gain practical experience in fine-tuning pre-trained vision models from TensorFlow/PyTorch and Hugging Face for image recognition tasks.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What are the advantages of transfer learning compared to training a CNN from scratch?

Transfer learning allows a model to reuse features that have already been learned from a large dataset such as ImageNet. In our experiment, ResNet-50 was used as the pre-trained feature extractor, so the model did not have to learn basic features such as edges, textures, shapes, and patterns from the beginning. This reduces training time and computational requirements compared with building and training a CNN completely from scratch. It can also achieve high accuracy even when the target dataset is relatively small. Therefore, transfer learning provides an efficient way to obtain strong image-classification performance with less training effort.

Advantage	Description
Reduced training time	Existing features are reused instead of learned from scratch.
Higher accuracy	Pre-trained models contain useful visual features learned from large datasets.
Less data required	Good performance can be obtained without extremely large target datasets.
Lower computational cost	Fewer parameters need to be trained.
Better generalization	Learned features can be transferred to related image-classification tasks.

2. Why are pre-trained models able to achieve higher accuracy with fewer training epochs? Pre-trained models already contain useful knowledge obtained from training on large and diverse datasets. For example, our ResNet-50 model had already learned important visual features before being adapted for the Cats and Dogs classification task. During transfer learning, the feature extraction layers were frozen and only the newly added classification layers needed to learn the specific Cats-vs-Dogs task. Because the model starts with meaningful visual representations instead of random weights, it can reach high accuracy within a small number of epochs. In our experiment, the TensorFlow transfer learning model reached approximately 97.85% test accuracy after only 5 training epochs.

3. Compare the performance of the transfer learning model with the CNN implemented in the previous experiment.

The CNN implemented in the previous experiment was trained from scratch using the CIFAR-10 dataset and achieved a final test accuracy of 70.36% after 10 epochs. In comparison, the TensorFlow ResNet-50 transfer learning model used in the current experiment achieved 97.85% test accuracy after only 5 epochs, while the Hugging Face ResNet-50 achieved 98.55% test accuracy. Therefore, the transfer learning models demonstrated substantially higher accuracy with fewer training epochs.

The improved performance is mainly because ResNet-50 starts with features learned from a large pre-trained dataset, whereas the CNN from the previous experiment had to learn its features from the beginning. The results show that transfer learning can provide a significant performance advantage when using a powerful pre-trained architecture. However, the datasets are different—CIFAR-10 for the previous CNN and Cats-vs-Dogs for the current transfer-learning experiment—so the accuracy values should be treated as a practical comparison rather than a strictly controlled benchmark.

Model	Dataset	Epochs	Test Accuracy
Custom CNN	CIFAR-10	10	70.36%
TensorFlow ResNet-50	Cats vs Dogs	5	97.85%
Hugging Face ResNet-50	Cats vs Dogs	—	98.55%

4. What is the purpose of freezing layers during transfer learning?

Freezing layers means preventing the weights of selected pre-trained layers from being updated during training. In our experiment, the ResNet-50 feature extraction layers were frozen because these layers had already learned useful general visual features from the original training data. This allowed the newly added classification layers to learn specifically how to distinguish between cats and dogs without changing the existing feature representations. Freezing layers also reduces the number of trainable parameters, which makes training faster and reduces computational requirements. It is particularly useful when the target dataset is smaller than the dataset used to originally train the pre-trained model.

5. How can Hugging Face simplify the deployment and experimentation of vision models?

Hugging Face provides a large collection of pre-trained models that can be directly loaded and used through libraries such as Transformers. In our experiment, we were able to load a pre-trained ResNet-50 model, use its image processor, modify it for two classes, and perform image classification on the Cats and Dogs dataset. This avoids having to manually implement the complete architecture and preprocessing pipeline of the model. Hugging Face also provides standardized model configurations and pre-trained weights, making it easier to experiment with different vision models. Therefore, it simplifies the process of developing, testing, comparing, and eventually deploying modern vision models.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		